

FoodLink AI — Use Case Scenarios

- **Project:** FoodLink AI (repository `UCS503P-202627-FoodBridge-AI`)
- **Course:** UCS503P — Software Engineering
- **Companion documents:** `FoodLink-UML.md` (full UML package) · `diagrams/01-use-case.puml` (Use Case Diagram)

1. About This Document

This document expands every actor-driven use case of the FoodLink AI Use Case Diagram into a structured scenario. The identifiers and names below are taken verbatim from the diagram, so `UC-07 Accept Donation` here is the same ellipse labelled *Accept Donation* there.

Each scenario was written against the implementation, not against the project description. Flows, guards and error responses correspond to actual code paths in `code/foodlink/`; nothing is described that the system does not do.

1.1 Status labels

Every scenario carries a **Status** field with one of two values:

LABEL	MEANING
Implemented — API and portal	Reachable by an end user through the web application today.
Implemented — API only	Present, authorised and covered by automated tests in the backend, but not yet offered by any screen in the web portal.

Nothing in this document is planned, proposed or backlogged work. Where a capability has a known limitation, it is stated inside the scenario rather than left implicit.

1.2 Actors

ACTOR	BASIS IN THE IMPLEMENTATION
Guest	Any caller without a bearer token. Only registration and login are reachable.
Registered User	Generalised actor for any authenticated, active account — the behaviour every role shares.
Donor	<code>UserRole.donor</code> — posts surplus food.
Recipient Organisation	<code>UserRole.ngo</code> , bound to a <code>Recipient</code> record — accepts donations, publishes needs.
Volunteer Courier	<code>UserRole.volunteer</code> , bound to a <code>Volunteer</code> record — collects and delivers.
Platform Administrator	<code>UserRole.admin</code> — vouches for organisations, manages accounts, may stand in on the lifecycle.

1.3 Conventions

- **Preconditions** state what must already be true; they are enforced by the server, not merely assumed of the user interface.
- **Alternative / Exception Flows** are labelled **A1**, **A2**, ... and give the HTTP status the system actually returns.
- A recurring pattern: where a caller holds the right *role* but is not the party a record belongs to, FoodLink answers **404 Not Found**, not 403. The record is re-read through the caller's own read scope, so an outsider cannot even confirm it exists.

2. Account and Access

UC-01 Register Account

Primary actor	Guest
Goal	Obtain a FoodLink account and an access token for one of the three self-service roles.
Preconditions	The email address is not already registered. The request is within the per-address rate limit.
Postconditions	The account exists and is active, and the caller holds a signed access token. A recipient organisation created this way is unverified , so it cannot yet be matched or accept donations.
Status	Implemented — API and portal

Main Success Scenario

1. Guest chooses a role: donor, recipient organisation, or volunteer courier.
2. Guest supplies name, email and a password of at least eight characters, and optionally an organisation name and phone number.
3. System confirms no account exists for that email address.
4. System hashes the password with bcrypt and creates the **User** record.
5. «include» *Provision Role Profile*: for a recipient account the system creates a **Recipient** organisation with `is_verified = false`; for a courier account it creates a **Volunteer** record.
6. System issues a signed access token and returns it with the new account (HTTP 201).

Alternative / Exception Flows

- **A1 — Email already registered.** HTTP 409; no account is created.
- **A2 — Administrator role requested.** Rejected by request-schema validation (HTTP 422). **admin** is not an accepted value, so registration can never mint an administrator.
- **A3 — Password shorter than eight characters, or malformed email.** HTTP 422.
- **A4 — Rate limit exceeded.** The request is refused before any account work is done.

Note on scope. The API also accepts an organisation type, address, coordinates and capacity at registration to seed the recipient record. The web sign-up form does not collect them, so an organisation registered through the portal must complete its profile afterwards (UC-20).

UC-03 Sign In

Primary actor	Guest
Goal	Exchange credentials for a bearer token and reach the portal for the account's role.
Preconditions	The account exists and is active. The request is within the per-address rate limit.
Postconditions	The client holds an access token whose expiry was fixed at issue; the portal has redirected to the home path for that role.
Status	Implemented — API and portal

Main Success Scenario

1. Guest submits an email address and password.
2. System looks up the account by lower-cased email.
3. System verifies the password against the stored bcrypt hash.
4. System confirms the account is active.
5. System issues an access token carrying the account id, role and expiry, and returns it with the account.
6. The portal redirects to the home path for the account's role.

Alternative / Exception Flows

- **A1 — Unknown email or wrong password.** HTTP 401 with a single message for both cases, so the response does not reveal which addresses hold accounts.
- **A2 — Account deactivated.** HTTP 403, directing the user to contact an administrator. The check happens at login rather than on a later request, so the person sees the real reason.
- **A3 — Rate limit exceeded.** The attempt is refused.

UC-04 Manage Account & Password

Primary actor	Registered User
Goal	Keep one's own contact details correct and rotate one's own password.
Preconditions	The caller is authenticated.
Postconditions	The account's name, organisation label or phone number is updated, and/or the password hash is replaced. Role, email and active status are unchanged.
Status	Implemented — API and portal

Main Success Scenario

1. User opens the profile screen for their portal.
2. User revises name, organisation label or phone number, and saves.
3. System applies only the supplied fields and returns the updated account.
4. To change the password, the user supplies the current password and a new one of at least eight characters.

5. System verifies the current password, replaces the hash, and confirms.

Alternative / Exception Flows

- **A1 — Current password incorrect.** HTTP 401; the password is unchanged.
- **A2 — New password shorter than eight characters.** HTTP 422.
- **A3 — Caller attempts to change role, email or active status.** Those fields are absent from the self-service schema and are silently not applied; they belong to an administrator (UC-26).

Why this exists. An account created by an administrator starts on a password somebody else chose. Without a self-service change, the first thing such an account would do is share a credential it could not rotate.

3. Donation Lifecycle

UC-05 Post Donation

Primary actor	Donor (<i>an administrator may also post</i>)
Goal	Publish surplus food so recipient organisations can claim it before it spoils.
Preconditions	The caller is authenticated as a donor or administrator.
Postconditions	The donation is in the open pool — AVAILABLE , or MATCHED if a candidate was found — and visible to every recipient organisation. No organisation is bound to it: a match is a suggestion, not a claim.
Status	Implemented — API and portal

Main Success Scenario

1. Donor supplies food name, category, quantity, unit, storage type, description, pickup location with coordinates, and a pickup deadline.
2. System confirms the deadline lies in the future.
3. System stores the donation with status **AVAILABLE** .
4. System appends a server-stamped status event recording **AVAILABLE** .
5. «include» *Rank Recipient Organisations*: the system scores the donation against every organisation that is verified, has coordinates, and lies inside the service radius.
6. If at least one organisation is eligible, the system freezes the leading candidate's score onto the donation and appends a status event recording **MATCHED** , noting the organisation's name.
7. System returns the stored donation.

Alternative / Exception Flows

- **A1 — Pickup deadline already past.** HTTP 422; nothing is stored.
- **A2 — Payload fails validation.** HTTP 422.
- **A3 — No eligible organisation.** The donation stays **AVAILABLE** with no frozen score. This is a normal outcome, not an error — the donation remains open for any organisation to accept.

UC-06 Browse Available Donations

Primary actor	Recipient Organisation
Goal	See what is currently on offer, and how well each offer suits <i>this</i> kitchen.
Preconditions	The caller is authenticated as a recipient organisation account.
Postconditions	None — this is a read. No donation changes state.
Status	Implemented — API and portal

Main Success Scenario

1. Organisation opens the available-donations screen.
2. System applies the read scope: every donation that is **AVAILABLE** or **MATCHED** , plus every donation already bound to this organisation.
3. «*extend*» *Show Compatibility Score*: for each donation still open to acceptance, the system scores the pairing against **this** organisation and attaches the headline percentage together with its five-criterion breakdown as one object.
4. The portal lists the donations ordered by pickup deadline.

Alternative / Exception Flows

- **A1 — Account has no organisation record yet.** The open pool is still returned, but no compatibility score is attached.
- **A2 — Donation no longer open to acceptance.** No live score is attached; the frozen score is shown instead, because that is the figure the accepting organisation actually decided on.

UC-07 Accept Donation

Primary actor	Recipient Organisation (<i>an administrator may accept on an organisation's behalf</i>)
Goal	Commit the organisation to receiving a donation.
Preconditions	The donation is AVAILABLE or MATCHED . The caller's organisation has been verified by an administrator.
Postconditions	The donation is ACCEPTED and bound to the organisation. It leaves the open pool and appears to couriers as an unclaimed pickup.
Status	Implemented — API and portal

Main Success Scenario

1. Organisation selects a donation from the open pool and accepts it.
2. System confirms the move to **ACCEPTED** is legal from the donation's current state.
3. System confirms the caller's role may set **ACCEPTED** .
4. System resolves the accepting organisation from the caller's own account.

5. System confirms the organisation is verified.
6. System binds the organisation to the donation, increments its accepted count, and «include» *Rank Recipient Organisations* to freeze the score this decision was made on.
7. System appends a status event recording **ACCEPTED**.

Alternative / Exception Flows

- **A1 — Organisation not verified.** HTTP 403. The donation stays in the pool. Verification is the platform's only check that a real organisation stands behind an account before food is handed to it.
- **A2 — Another organisation accepted first.** The donation has left the open pool, so the transition is refused with HTTP 409.
- **A3 — Caller names a different organisation.** HTTP 403. An organisation may only accept as itself; naming an arbitrary organisation is an administrator action (UC-27).
- **A4 — Caller's account has no organisation record.** HTTP 422.

UC-08 Claim Pickup	
Primary actor	Volunteer Courier
Goal	Take responsibility for collecting and delivering an accepted donation.
Preconditions	The donation is ACCEPTED with no courier assigned. The caller's account has a courier profile.
Postconditions	The courier is bound to the pickup, which moves to VOLUNTEER_ASSIGNED . No other courier can claim it while the binding stands.
Status	Implemented — API and portal

Main Success Scenario

1. Courier sees the pickup in the unclaimed queue.
2. Courier claims it.
3. System issues a single conditional update that binds the courier **only if** the donation is still in the expected state and is either unclaimed or already this courier's.
4. Exactly one row is affected — the claim is won.
5. System appends a status event recording **VOLUNTEER_ASSIGNED**.

Alternative / Exception Flows

- **A1 — Another courier won the race.** The conditional update affects no rows. The system re-reads the donation and answers HTTP 409, "Another courier has already claimed this pickup."
- **A2 — Donation changed state in the meantime.** HTTP 409 naming the illegal move.
- **A3 — The same courier sends a duplicate request.** The claim succeeds once; the state guard prevents a second status event being appended.
- **A4 — Account has no courier profile.** HTTP 422. This is also why an administrator cannot claim a pickup, although they may drive every other transition (UC-27).

Why the check and the write are one operation. Two couriers can both read "unclaimed" and both decide they may have it. Only one can satisfy the condition *as the database applies the write*, which is where the platform puts it.

UC-09 Record Collection

Primary actor	Volunteer Courier
Goal	Record that the food has been collected from the donor, leaving an attributable timestamp.
Preconditions	The donation is VOLUNTEER_ASSIGNED and the caller is the courier assigned to <i>this</i> donation.
Postconditions	The donation is PICKED_UP ; the collection time is now part of the status history.
Status	Implemented — API and portal

Main Success Scenario

1. Courier collects the food from the donor.
2. Courier records the collection in the task list.
3. System confirms the move **VOLUNTEER_ASSIGNED** → **PICKED_UP** is legal and that the caller's role may drive it.
4. System re-reads the donation through the caller's own read scope, confirming this courier is the assignee.
5. System appends a status event recording **PICKED_UP** .

Alternative / Exception Flows

- **A1 — A courier who is not the assignee attempts the step.** HTTP 404 rather than 403 — a 403 would confirm the donation exists.
- **A2 — Donation not yet claimed, or already collected.** HTTP 409.

UC-10 Record Delivery

Primary actor	Volunteer Courier
Goal	Record that the food has been handed over at the receiving organisation.
Preconditions	The donation is PICKED_UP and the caller is the courier assigned to <i>this</i> donation.
Postconditions	The donation is DELIVERED . The acceptance-to-delivery interval is now derivable from the status history and feeds the median handover metric.
Status	Implemented — API and portal

Main Success Scenario

1. Courier hands the food over at the kitchen.
2. Courier records the delivery.
3. System confirms the move **PICKED_UP** → **DELIVERED** is legal and that the caller's role may drive it.

4. System re-reads the donation through the caller's own read scope, confirming this courier is the assignee.
5. System appends a status event recording **DELIVERED** .

Alternative / Exception Flows

- **A1 — A courier who is not the assignee attempts the step.** HTTP 404.
- **A2 — Delivery recorded before collection.** HTTP 409; **DELIVERED** is reachable only from **PICKED_UP** .

UC-11 Confirm Completion

Primary actor	Recipient Organisation
Goal	Confirm the food was received, closing the donation successfully.
Preconditions	The donation is DELIVERED and bound to the caller's organisation.
Postconditions	The donation is COMPLETED , which is terminal. The organisation's completion count and the courier's delivery count both increase, which changes future match rankings.
Status	Implemented — API and portal

Main Success Scenario

1. Organisation confirms receipt of the donation.
2. System confirms the move **DELIVERED** → **COMPLETED** is legal and that the caller's role may drive it.
3. System re-reads the donation through the caller's read scope, confirming this is the accepting organisation.
4. System increments the organisation's completed-donation count and the courier's completed-delivery count.
5. System appends a status event recording **COMPLETED** .

Alternative / Exception Flows

- **A1 — An unrelated organisation attempts completion.** HTTP 404.
- **A2 — Donation not yet delivered.** HTTP 409.

UC-12 Release Pickup

Primary actor	Recipient Organisation (<i>or an administrator standing in</i>)
Goal	Return a pickup to the courier queue when the assigned courier cannot complete it.
Preconditions	The donation is VOLUNTEER_ASSIGNED and bound to the caller's organisation.
Postconditions	The donation is ACCEPTED again with no courier, and is claimable by any courier including the original one. The organisation's accepted count is not incremented again and the frozen match score is not re-frozen — a release is not a second acceptance.
Status	Implemented — API only

Main Success Scenario

1. Organisation releases the pickup.
2. System confirms the caller owns this donation. Ownership is required here even though ordinary acceptance is open to all, because a release acts on a donation that is already somebody's.
3. System clears the courier assignment so the pickup becomes visible and claimable again.
4. System appends a status event recording **ACCEPTED**.

Alternative / Exception Flows

- **A1 — A peer organisation attempts the release.** HTTP 404; the binding is untouched, and in particular the peer cannot take ownership of the donation this way.
- **A2 — The assigned courier tries to release themselves.** Refused on role — **ACCEPTED** is not a transition a courier may drive.

UC-13 Cancel Donation

Primary actor	Donor (<i>or an administrator</i>)
Goal	Withdraw a donation that can no longer be given.
Preconditions	The caller posted this donation. The donation is AVAILABLE , MATCHED , ACCEPTED , VOLUNTEER_ASSIGNED or PICKED_UP .
Postconditions	The donation is CANCELLED , which is terminal. It counts towards neither the rescue rate nor the expiry-loss rate.
Status	Implemented — API only

Main Success Scenario

1. Donor cancels the donation.
2. System confirms the move to **CANCELLED** is legal from the current state and that the caller's role may drive it.
3. System re-reads the donation through the donor's read scope, confirming ownership.
4. System appends a status event recording **CANCELLED**.

Alternative / Exception Flows

- **A1 — A different donor attempts cancellation.** HTTP 404.
- **A2 — Donation already delivered, completed, expired or cancelled.** HTTP 409; cancellation is not reachable from those states.
- **A3 — A recipient organisation or courier attempts cancellation.** HTTP 403 on role.

4. Matching and Explainability

UC-16 Review Match Explanation

Primary actor	Donor
Goal	Understand <i>why</i> a donation was matched to a particular organisation, rather than being given a bare percentage.
Preconditions	The caller is entitled to read the donation.
Postconditions	None — this is a read.
Status	Implemented — API and portal

Main Success Scenario

1. Donor opens the match analysis for one of their donations.
2. System applies the same read scope the donation itself uses — seeing the reasoning is seeing the donation.
3. System ranks the eligible organisations and returns, for each, the overall score together with its five component scores — distance, quantity fit, capacity headroom, deadline feasibility and reliability — plus the human-readable reasons behind them.
4. The portal presents the leading match with its breakdown.

Alternative / Exception Flows

- **A1 — Donation not readable by the caller.** HTTP 404.
- **A2 — No eligible organisation.** An empty ranking is returned, not an error.
- **A3 — Reader is not the organisation being scored.** The pairing is still ranked, but it is scored from a deliberately coarsened position and the true distance is withheld rather than approximated. A donor chooses the donation's own location, so publishing exact distances would let a kitchen's withheld address be recovered by measuring from several chosen points.

Why a transparent heuristic. The ranking is a published weighted sum over five normalised criteria, not a learned model, precisely so that a score can always be taken apart and checked by hand.

5. Standing Demand

UC-17 Post Standing Requirement

Primary actor	Recipient Organisation
Goal	Publish an ongoing need so donors can see demand before matching supply to it.
Preconditions	The caller's account is bound to a recipient organisation.
Postconditions	The need is active on the organisation's own board, and — if the organisation is verified — on every donor's needs board.
Status	Implemented — API and portal

Main Success Scenario

1. Organisation supplies food type, quantity needed, unit, beneficiary count, urgency, whether the need recurs daily, and free-text notes.
2. System creates the requirement against the caller's own organisation, active by default.
3. System returns the requirement, carrying the posting organisation's verification state so a donor board can label the card accordingly.

Alternative / Exception Flows

- **A1 — An administrator attempts to post.** The role gate admits them, but an administrator account is not bound to any organisation, so the system answers HTTP 422.
- **A2 — Invalid quantity or empty food type.** HTTP 422.

Note. A standing requirement is a demand signal for donors. It is **not** an input to the matcher, which scores donations against organisations only.

UC-18 Revise, Retire or Reopen Need

Primary actor	Recipient Organisation
Goal	Keep the needs board honest: correct a need, take a met need off the board, or put a retired need back.
Preconditions	The requirement belongs to the caller's organisation.
Postconditions	A retired need leaves the donor board immediately; the row is kept either way, so demand history survives and the need can be reopened later.
Status	Implemented — API and portal

Main Success Scenario

1. Organisation edits a need, or marks it retired or reopened.
2. System resolves the requirement **within the caller's organisation only**.

3. System applies the supplied fields. Marking it inactive retires it; marking it active puts it back on the board.
4. System returns the updated requirement.

Alternative / Exception Flows

- **A1 — Another organisation's requirement identifier.** HTTP 404, indistinguishable from an identifier that never existed.
- **A2 — An explicit null for a field.** The field is left alone rather than cleared: no requirement field is nullable, so null cannot mean "clear this".
- **A3 — A donor or courier attempts the edit.** HTTP 403 on role.

One flag, two meanings. A need that has been met and a need that no longer applies are both simply off the board; the model carries a single lifecycle flag rather than a separate "fulfilled" state.

UC-19 Browse Needs Board

Primary actor	Donor
Goal	See what organisations currently need, in order to decide what to cook or set aside.
Preconditions	The caller is authenticated as a donor.
Postconditions	None — this is a read.
Status	Implemented — API and portal

Main Success Scenario

1. Donor opens the needs board.
2. System returns active requirements posted by **verified** organisations, newest first.
3. The portal renders each need with its organisation, urgency and beneficiary count.

Alternative / Exception Flows

- **A1 — Donor asks for retired needs as well.** The request is ignored for donors. A retired need is not an offer and a donor has no action on one, so this is a permission rather than a filter the caller may choose.
- **A2 — A courier calls the same endpoint.** An empty list. Standing demand is not an input to a courier's work.
- **A3 — No verified organisation has posted a need.** An empty board, not an error.

Why verification gates the board. Verification is the same gate the matcher applies, so a donor is never shown a need from an organisation that could not have received the donation anyway.

6. Profiles and Monitoring

UC-20 Maintain Organisation Profile

Primary actor	Recipient Organisation
Goal	Complete or correct the organisation's profile so it can be matched accurately.
Preconditions	The caller is a recipient organisation account with an organisation record.
Postconditions	The revised capacity and location feed the ranking heuristic. Verification status is unchanged.
Status	Implemented — API and portal

Main Success Scenario

1. Organisation opens its profile.
2. Organisation revises name, type, address, capacity, contact person or phone.
3. System applies only the supplied fields and returns the updated record.

Alternative / Exception Flows

- **A1 — Caller attempts to mark itself verified.** The field is absent from the update schema; an organisation cannot vouch for itself (see UC-24).
- **A2 — Account has no organisation record.** HTTP 422.

Known limitation. The API accepts latitude and longitude here, but the portal's profile form does not send them and the project carries no geocoder, so an organisation created entirely through the web interface has no coordinates — and an organisation without coordinates cannot be matched at all. Coordinates can currently be supplied only through the API or seed data.

UC-21 Manage Courier Availability

Primary actor	Volunteer Courier
Goal	Go on or off duty and keep one's base location current.
Preconditions	The caller is a courier account with a courier profile.
Postconditions	The courier's availability and base location are updated. Delivery count and rating are unchanged.
Status	Implemented — API and portal

Main Success Scenario

1. Courier opens their profile.
2. Courier toggles availability and/or edits their base location.
3. System applies the supplied fields and returns the updated profile.

Alternative / Exception Flows

- **A1 — Caller attempts to set delivery count or rating.** Those fields are absent from the update schema: they are earned through completed runs, so they are the server's to maintain, not the courier's.
- **A2 — Account has no courier profile.** HTTP 422.

Availability is advisory. Going off duty changes what organisations see on the roster; it does not remove the courier's existing pickups, and open pickups remain claimable.

UC-22 View Donations (role-scoped)

Primary actor	Registered User
Goal	See the donations relevant to one's own part in the process, and nothing beyond it.
Preconditions	The caller is authenticated.
Postconditions	None — this is a read.
Status	Implemented — API and portal

Main Success Scenario

1. User opens a dashboard or donation list.
2. System applies the read scope for the caller's role: consider, plus the donations bound to their own organisation. they are the courier for, in whatever state.
 - **Donor** — the donations they posted, and nothing else.
 - **Recipient Organisation** — the open pool every organisation is invited to
 - **Volunteer Courier** — pickups waiting for a courier, plus every donation
 - **Administrator** — unrestricted.
3. System optionally narrows further by status, or to the caller's own records.
4. The portal renders the list, ordered by pickup deadline.

Alternative / Exception Flows

- **A1 — Caller requests a donation outside their scope by identifier.** HTTP 404. The scope is applied inside the query, so an unauthorised identifier is indistinguishable from one that does not exist.
- **A2 — A role with no scope defined.** Nothing is returned. The system fails closed: a role added later reads nothing until it is explicitly given a scope.

UC-23 View Platform Metrics

Primary actor	Registered User
Goal	See the platform's measured impact and responsiveness.
Preconditions	The caller is authenticated.
Postconditions	None — this is a read.
Status	Implemented — API and portal

Main Success Scenario

1. User opens an impact or analytics screen.
2. System derives the figures from the append-only status history: total and completed donations, meals delivered, active and expired donations, organisation and courier counts, median time-to-claim, median handover time, rescue rate and expiry-loss rate.
3. The portal renders the figures.

Alternative / Exception Flows

- **A1 — Not enough history to compute a median or rate.** Those figures are returned as absent rather than as zero, so an empty platform is not reported as a failing one.

Attributability. Every figure is computed from status events stamped by the server at the moment of each transition. Nothing here is self-reported, which is what makes the evaluation metrics defensible. Note that the metrics are platform-wide and are not narrowed per role.

7. Administration

UC-24 Verify or Revoke Organisation

Primary actor	Platform Administrator
Goal	Vouch for a real organisation, or withdraw that endorsement.
Preconditions	The caller holds an administrator token.
Postconditions	The organisation's verification state is changed. Verification decides whether it is ranked, whether it may accept donations, and whether its standing needs reach donors.
Status	Implemented — API and portal

Main Success Scenario

1. Administrator reviews the organisation directory and the count awaiting verification.
2. Administrator marks an organisation verified.
3. System records the verification and returns the organisation.

Alternative / Exception Flows

- **A1 — Unknown organisation identifier.** HTTP 404.
- **A2 — Revocation.** The same operation in reverse withdraws verification. The organisation immediately drops out of the matcher and can no longer accept donations; donations it has already accepted are unaffected.
- **A3 — A non-administrator calls the route.** HTTP 403 at the router-level gate that guards every administration path.

Why a person decides. Verification is a human judgement — somebody confirmed this kitchen is real and is where it claims to be — so it is an administrator action rather than something an organisation asserts about itself.

UC-25 Run Expiry Sweep

Primary actor	Platform Administrator
Goal	Move donations nobody claimed before their deadline into a terminal state, so losses are recorded rather than left looking available.
Preconditions	The caller holds an administrator token.
Postconditions	Overdue unclaimed donations are EXPIRED and counted in the expiry-loss rate.
Status	Implemented — API and portal

Main Success Scenario

1. Administrator triggers the sweep.
2. System selects every donation still **AVAILABLE** or **MATCHED** whose pickup deadline has passed.
3. For each, the system appends a status event recording **EXPIRED**, noting that the deadline passed with no recipient.
4. System reports how many donations expired.

Alternative / Exception Flows

- **A1 — Nothing is overdue.** The sweep reports zero. Not an error.
- **A2 — A donation has already been accepted.** It is left untouched; only unclaimed donations expire.

No scheduler exists. The sweep runs only when a person triggers it. Its status events carry **no actor**, because no person performed the transition.

UC-26 Manage User Accounts

Primary actor	Platform Administrator
Goal	Create accounts of any role, and suspend, restore, rename or re-role existing ones.
Preconditions	The caller holds an administrator token.
Postconditions	The account exists or is changed as requested. At least one active administrator always remains.
Status	Implemented — API only

Main Success Scenario

1. Administrator supplies name, email, password and role for a new account — including, uniquely on this path, the administrator role itself.
2. System confirms no account exists for that email and creates the account.
3. System mirrors the side effects of self-registration, so the account is not a second-class one missing its profile: a courier account gains a courier profile, and an organisation account gains an organisation record that is **already verified**, because an administrator creating it *is* the vouching.
4. To change an existing account, the administrator suspends, restores, renames or re-roles it, and the system applies the change.

Alternative / Exception Flows

- **A1 — Email already registered.** HTTP 409.
- **A2 — Administrator suspends or demotes their own account.** HTTP 409, refused.
- **A3 — The change would remove the last active administrator.** HTTP 409; the administrator must appoint another one first.
- **A4 — The very first administrator.** Cannot come from this use case, because calling it already requires being one. It is created from the command line instead.

Portal status. These operations are implemented, authorised and reachable through the API, but no screen in the web portal currently drives them.

UC-27 Override Lifecycle Transition

Primary actor	Platform Administrator
Goal	Act on any donation as a stand-in, so support staff can unblock a handover that the parties themselves cannot complete.
Preconditions	The caller holds an administrator token.
Postconditions	The donation reaches the requested state and the status event records the administrator as the actor.
Status	Implemented — API and portal

Main Success Scenario

1. Administrator opens a donation, which they may read regardless of who it belongs to.
2. Administrator drives a lifecycle transition on it — accepting on an organisation's behalf («*extend*» UC-07), releasing a pickup («*extend*» UC-12), recording collection or delivery, confirming completion, cancelling, or marking a donation expired.
3. System accepts the transition: the administrator role is admitted for every target state, and the ownership rule that constrains other roles does not narrow an administrator's unrestricted read scope.
4. System appends a status event attributing the change to the administrator.

Alternative / Exception Flows

- **A1 — The transition is illegal from the donation's current state.** HTTP 409. An administrator may bypass *who* drives a transition, never *which* transitions are legal.
- **A2 — Administrator tries to claim a pickup.** HTTP 422. Claiming binds the caller's own courier profile, and an administrator has none (see UC-08).
- **A3 — Administrator tries to post or edit a standing requirement.** HTTP 422: an administrator is not bound to any organisation (see UC-17).

Broad but not total. The override covers the donation lifecycle. It does not make an administrator a member of every role — two paths still refuse them, for structural reasons rather than policy ones.

8. Use Cases Intentionally Without a Scenario

Three use cases appear in the Use Case Diagram but are deliberately not given standalone scenarios, because no actor initiates them. Each is a behaviour of a use case that *is* initiated by an actor, and is documented as a numbered step inside it.

ID	NAME	WHY IT HAS NO SCENARIO	DOCUMENTED AS A STEP IN
UC-02	Provision Role Profile	« <i>include</i> » fragment — it always runs as part of registration and can never be invoked on its own.	UC-01, step 5
UC-14	Rank Recipient Organisations	« <i>include</i> » fragment — it always runs when a donation is posted and again when one is first accepted.	UC-05 step 5; UC-07 step 6
UC-15	Show Compatibility Score	« <i>extend</i> » fragment — conditional behaviour attached to browsing, never initiated by itself.	UC-06, step 3

This is why the twenty-seven use cases in the diagram yield twenty-four scenarios here. No use case has been dropped, renamed or invented: every identifier and name above matches the diagram exactly.

9. Traceability

SECTION	USE CASES	PRIMARY ACTOR
2 — Account and Access	UC-01, UC-03, UC-04	Guest, Registered User
3 — Donation Lifecycle	UC-05, UC-06, UC-07, UC-08, UC-09, UC-10, UC-11, UC-12, UC-13	Donor, Recipient Organisation, Volunteer Courier
4 — Matching and Explainability	UC-16	Donor
5 — Standing Demand	UC-17, UC-18, UC-19	Recipient Organisation, Donor
6 — Profiles and Monitoring	UC-20, UC-21, UC-22, UC-23	Recipient Organisation, Volunteer Courier, Registered User
7 — Administration	UC-24, UC-25, UC-26, UC-27	Platform Administrator

Verified against: `code/foodlink/models.py` (states and transitions), `routes/auth.py`, `routes/donations.py`, `routes/organisations.py`, `routes/admin.py`, `routes/metrics.py`, `matching.py`, `schemas.py`, and the lifecycle tests under `code/tests/`. Portal availability was confirmed against `frontend/src/App.tsx`, `lib/api.ts` and the relevant pages.